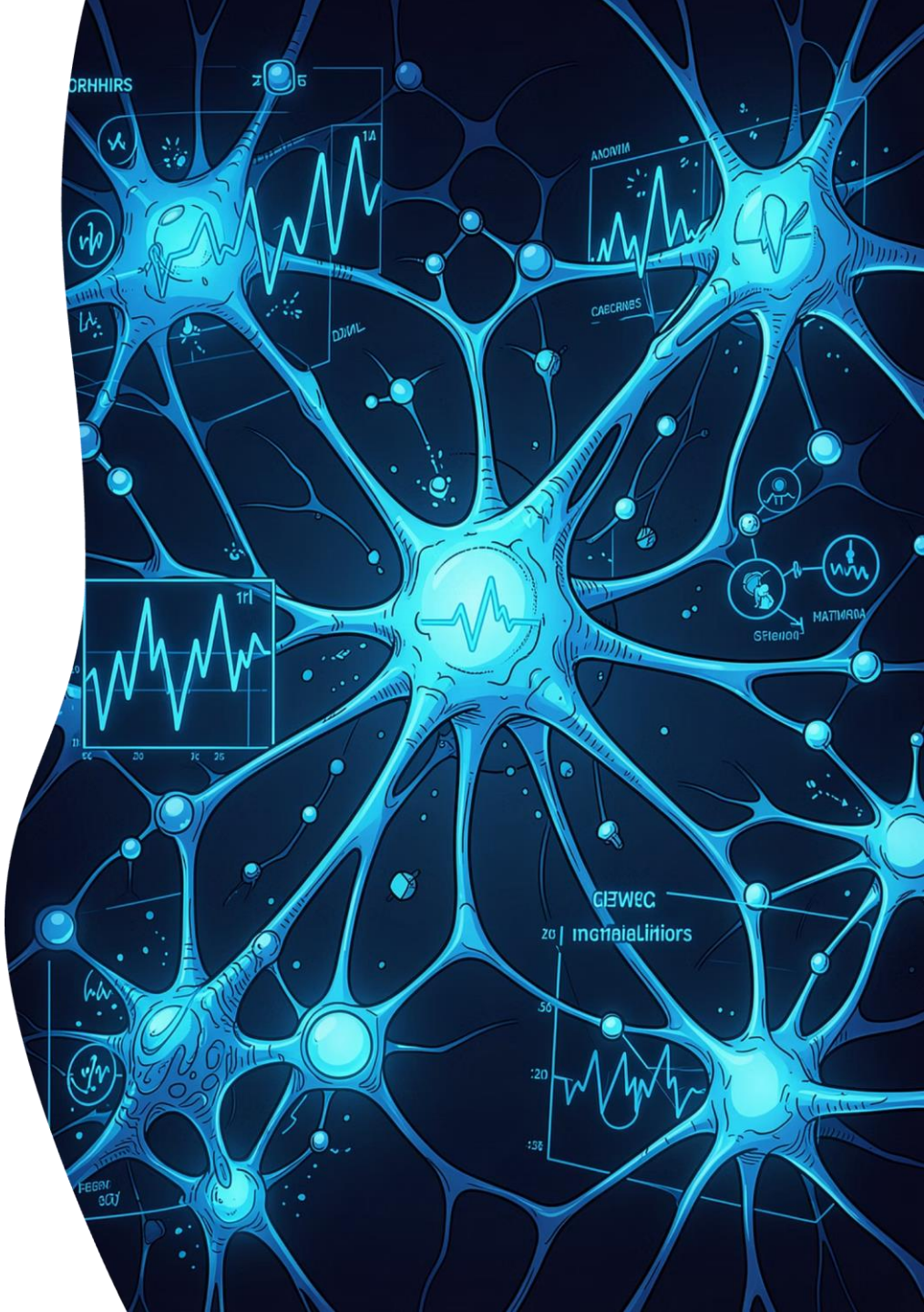
Diabetes Prediction

A complete machine learning pipeline for predicting diabetes diagnosis probability from patient health indicators — covering exploratory analysis, preprocessing, model training, and evaluation.

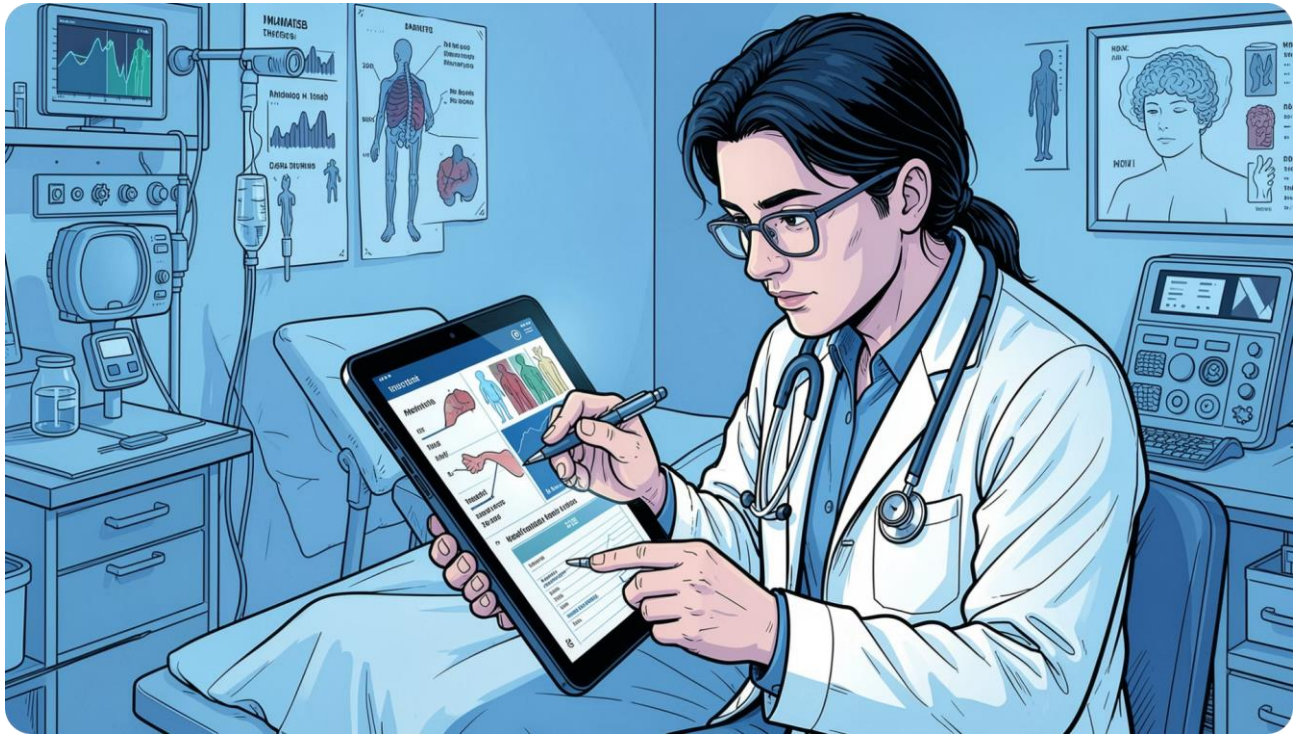
KAGGLE COMPETITION · PLAYGROUND SERIES S5E12

SANSONE LORENZO · UNIVERSITY OF BOLOGNA · 2025/2026



OVERVIEW

Objective & Dataset



Goal

Predict the **probability** of `diagnosed_diabetes` for each patient. Primary metric: **ROC-AUC** (Area Under the ROC Curve).

Data Source

Synthetically generated from the **Diabetes Health Indicators Dataset** — realistic distributions, privacy-preserving.

Files

- **train.csv** — 700,000 rows, 26 columns

PIPELINE

Full ML Workflow

01

Exploratory Analysis & Cleaning

Understand distributions, detect outliers, check class balance, correlations

02

Preprocessing

Encode categoricals, scale continuous features

03

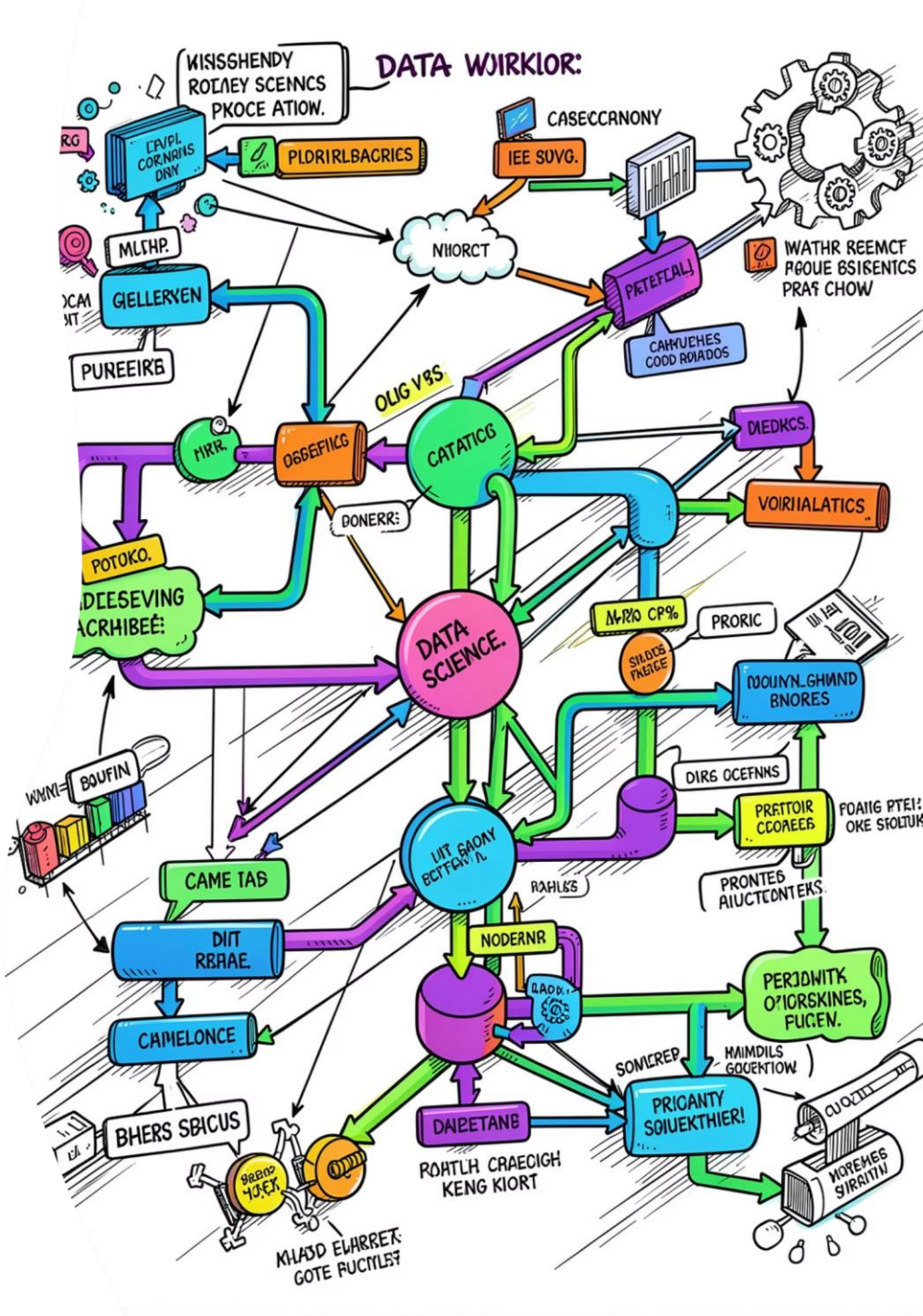
Model Training

GridSearchCV with 5-fold StratifiedKFold across 8 classifiers

04

Evaluation & Comparison

ROC-AUC on isolated hold-out test set



DATA EXPLORATION

Dataset at a Glance

700K

Rows

Synthetic patient profiles

24

Features

4 domains

0

Missing Values

Clean dataset

62%

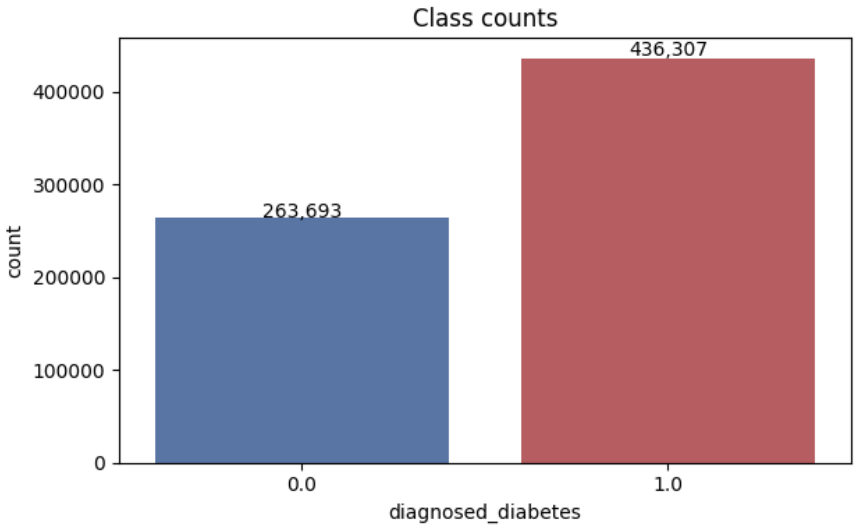
Positive Class

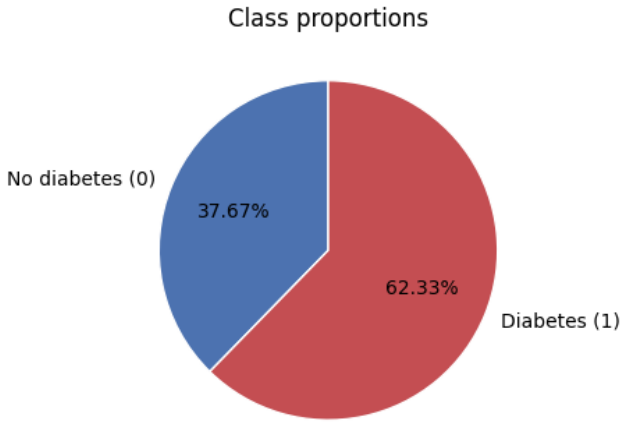
Mildly imbalanced

Features span **clinical measurements** (BMI, blood pressure, cholesterol, ...), **lifestyle habits** (diet, activity, sleep, ...), **demographics** (age, ethnicity, income, ...) and **medical history** (family diabetes, hypertension, cardiovascular, ...).

DATA EXPLORATION

Target Distribution & Class Balance





Class Proportions

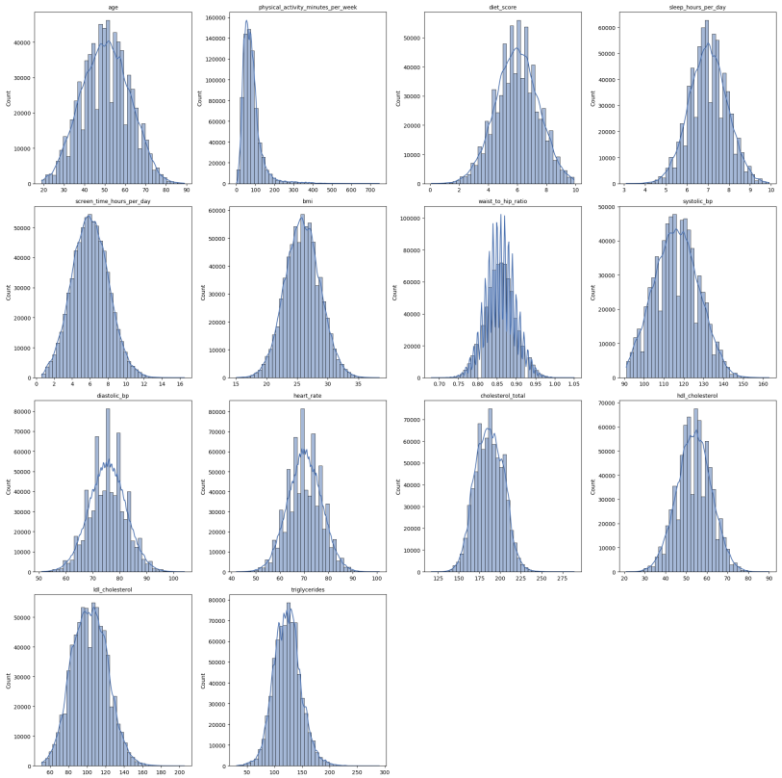
62.33% diagnosed with diabetes (positive class) vs **37.67%** without.
Positive/negative ratio: **1.65**.
The target is **mildly imbalanced** — stratified splits and class-weighting were considered, but stratification alone proved sufficient.

No duplicate rows detected. Zero NaN values across all 26 columns.

DATA EXPLORATION

Numerical Features Distributions

To understand the intrinsic shape, symmetry, and potential anomalies of our continuous features, a histogram with a Kernel Density Estimate (KDE) was plotted for each of the 14 numerical variables, using 40 bins.



Key Observations

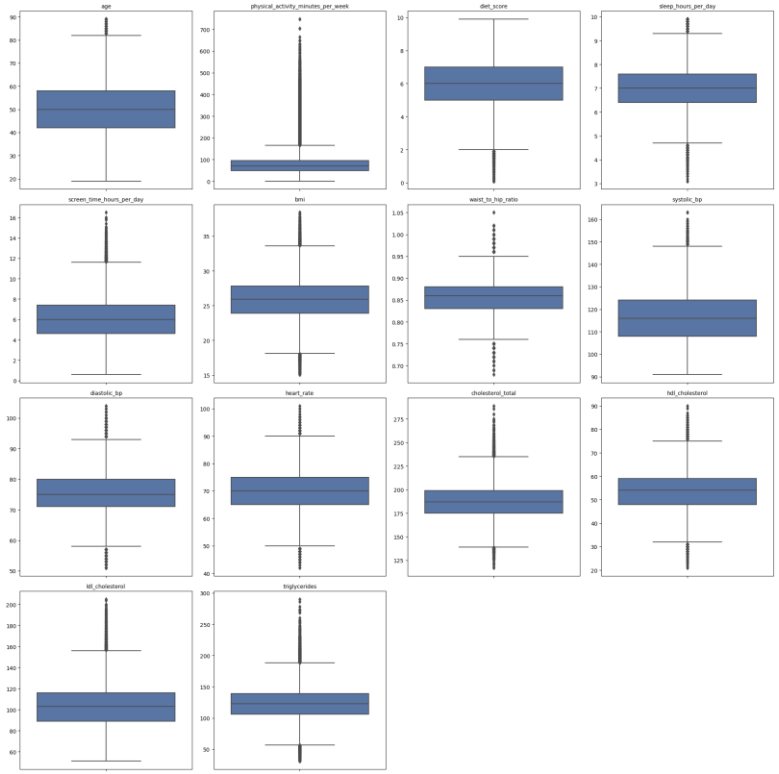
- Most clinical features (`bmi`, `systolic_bp`, `heart_rate`, `cholesterol` metrics) exhibit a roughly bell-shaped, near-Gaussian distribution.
- `physical_activity_minutes_per_week` is distinctly right-skewed with a prominent long tail, indicating a few patients with exceptionally high activity levels.
- `diet_score`, `sleep_hours_per_day`, and `screen_time_hours_per_day` generally appear symmetric.
- `age` spans a wide range from 19 to 89 years, with a relatively uniform distribution across adult age groups.

Takeaway: The majority of features are mostly well-behaved and near-Gaussian, which supports the effectiveness of StandardScaler for preprocessing. `physical_activity_minutes_per_week` remains the main exception, echoing its significance in outlier analysis.

DATA EXPLORATION

Outlier Detection (IQR Rule)

Outliers are flagged using the IQR rule: points falling outside [Q1 - 1.5 * IQR, Q3 + 1.5 * IQR].



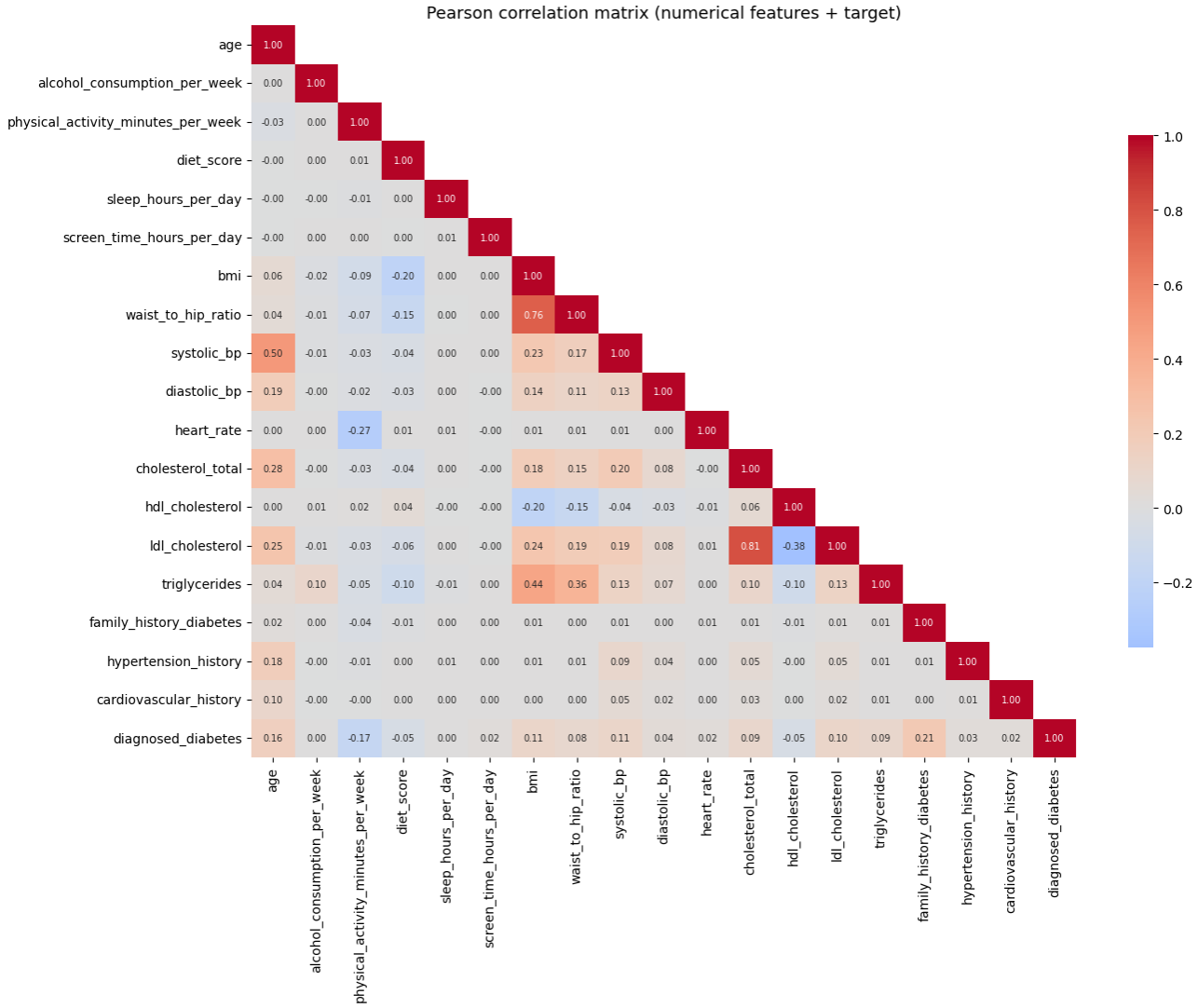
Key Observations

- **Heavy upper tails:** Pronounced in `physical_activity_minutes_per_week` (max value 747, far exceeding IQR upper bound), `triglycerides`, `cholesterol_total`, `ldl_cholesterol`, `systolic_bp`, and `bmi`.
- **Two-sided tails:** Features like `hdl_cholesterol`, `waist_to_hip_ratio`, and `heart_rate` exhibit significant outliers on both the lower and upper bounds.

Takeaway: This highlights the necessity of feature scaling for linear models (SVM, KNN), which are highly sensitive to extreme values. Tree-based models (Random Forest, XGBoost, CatBoost) are inherently more robust to outliers.

DATA EXPLORATION

Correlation with Target



Key Insights

All per-feature correlations are **weak** — no single feature is a strong standalone predictor.

- **Top positive:** family_history_diabetes (r=0.21), age (r=0.16), systolic_bp, BMI, LDL
- **Top negative:** physical_activity_minutes_per_week (r=−0.17), HDL, diet_score
- **Near-zero:** sleep_hours_per_day, alcohol_consumption_per_week

This explains the modest final ROC-AUC — the signal is distributed across many weak features.

PREPROCESSING

From Raw Data to Model-Ready Matrix

1

Drop ID + Target

Remove non-predictive row identifier

2

Encode Categoricals

Ordinal for education & income; one-hot for gender, ethnicity, smoking, employment

3

Split Data

67% train / 33% test, stratified

4

Scale Features

StandardScaler on continuous features (mean=0, std=1)

Feature matrix expanded from **24 → 35 columns** after one-hot encoding. StandardScaler outperformed RobustScaler and no-scaling in validation tests.

PREPROCESSING

Encoding of Categorical Features

To prepare categorical features for machine learning models, three distinct encoding strategies were applied based on the variable's nature.

Ordinal Encoding

For variables with an inherent, meaningful order, ordinal encoding was used to preserve their ranking information.

- `education_level`: No formal < Highschool < Graduate < Postgraduate
- `income_level`: Low < Lower-Middle < Middle < Upper-Middle < High

One-Hot Encoding

Purely nominal variables, where no natural order exists, were converted into binary (0/1) columns.

- `gender`
- `ethnicity`
- `smoking_status`
- `employment_status`

No Encoding Needed

Binary flags, already represented as 0s and 1s, required no further transformation.

- `family_history_diabetes`
- `hypertension_history`
- `cardiovascular_history`

The one-hot encoding process expanded the feature matrix from **24 raw features** to **35 columns**.

PREPROCESSING

Feature Scaling — StandardScaler

StandardScaler standardizes each continuous feature to mean 0 and standard deviation 1 ($z = (x - \mu) / \sigma$), mapping it onto the standard normal distribution.

Why use it?

Features with different ranges and units can bias scale-sensitive algorithms (KNN, SVM, Logistic Regression, Neural Networks). Scaling ensures all features contribute equally by putting them on a **comparable scale**.

Scope of application

Applied exclusively to **continuous numerical features**. Binary flags and one-hot encoded columns are intentionally left unchanged, preserving their intrinsic categorical meaning.

Critical Rule: Prevent Data Leakage

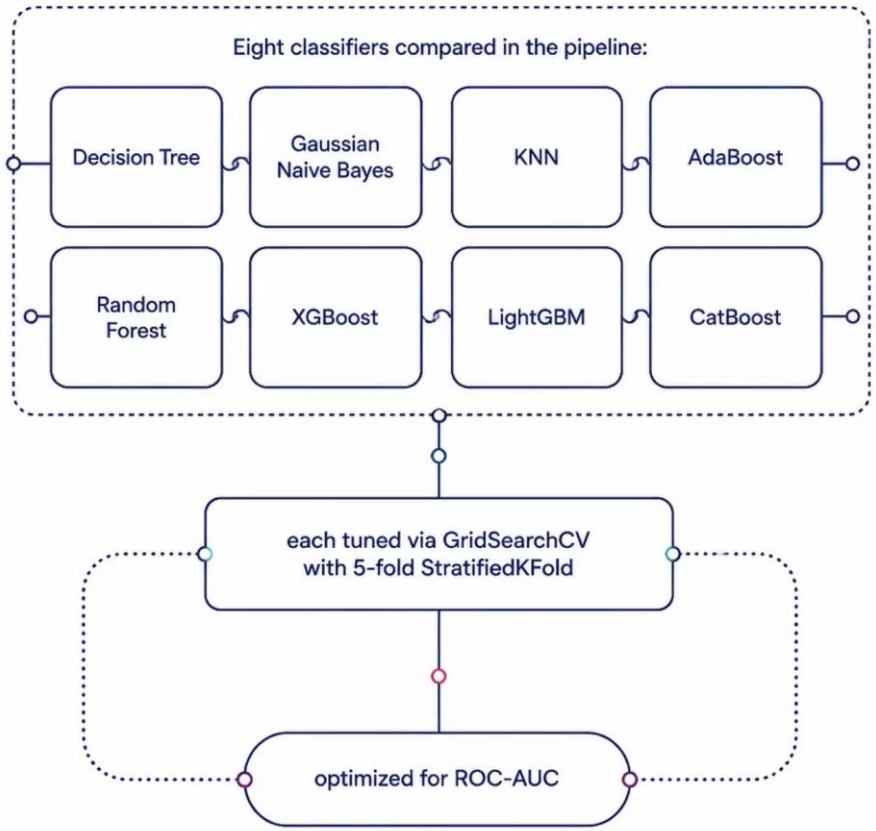
The scaler must be fit only on the training set. This fitted scaler is then used to transform **both the training and test** data, preventing information from the test set from influencing the training process.

Note on Tree-Based Models

Tree-based models (Random Forest, XGBoost, CatBoost) are **scale-invariant** and can perform well on unscaled data. For this project, they were evaluated using both scaled and unscaled features for completeness.

MODEL TRAINING

8 Classifiers, GridSearchCV & ROC-AUC



Training Setup

- **Scoring:** ROC-AUC (robust to class imbalance)
- **CV:** 5-fold StratifiedKFold (preserves class ratio)
- **Hold-out:** Test set (~231K rows) touched **once** at the end

Parameter Grids

Ranging from **10 combinations** (Naive Bayes) to **54** (XGBoost, LightGBM). All models tuned with identical CV strategy for fair comparison.

RESULTS

Evaluation: Boosting Models Dominate

ROC-AUC Rankings (Test Set)

Results for scoring "roc_auc"

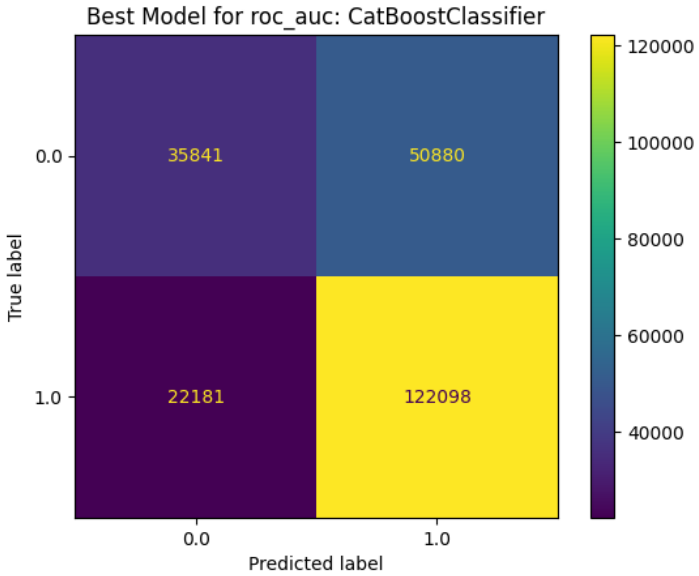
	model	best_params	accuracy	precision_macro	recall_macro	f1_macro	roc_auc
7	CatBoostClassifier	{'auto_class_weights': 'None', 'depth': 6, 'iterations': 400, 'learning_rate': 0.2}	0.684	0.662	0.630	0.632	0.726
6	LGBMClassifier	{'learning_rate': 0.1, 'max_depth': -1, 'n_estimators': 300, 'num_leaves': 31}	0.683	0.660	0.630	0.633	0.725
5	XGboost	{'learning_rate': 0.05, 'max_depth': 6, 'min_child_weight': 10, 'n_estimators': 600}	0.684	0.661	0.631	0.634	0.725
3	AdaBoost	{'learning_rate': 1.5, 'n_estimators': 50}	0.667	0.641	0.606	0.605	0.701
4	Random forest	{'max_depth': 9, 'n_estimators': 50}	0.660	0.646	0.574	0.555	0.696
0	Decision Tree	{'class_weight': None, 'max_depth': 9}	0.662	0.633	0.601	0.600	0.692
1	Gaussian Naive Bayes	{'var_smoothing': 0.001}	0.625	0.616	0.623	0.615	0.676
2	K Nearest Neighbor	{'n_neighbors': 6}	0.576	0.560	0.563	0.560	0.585

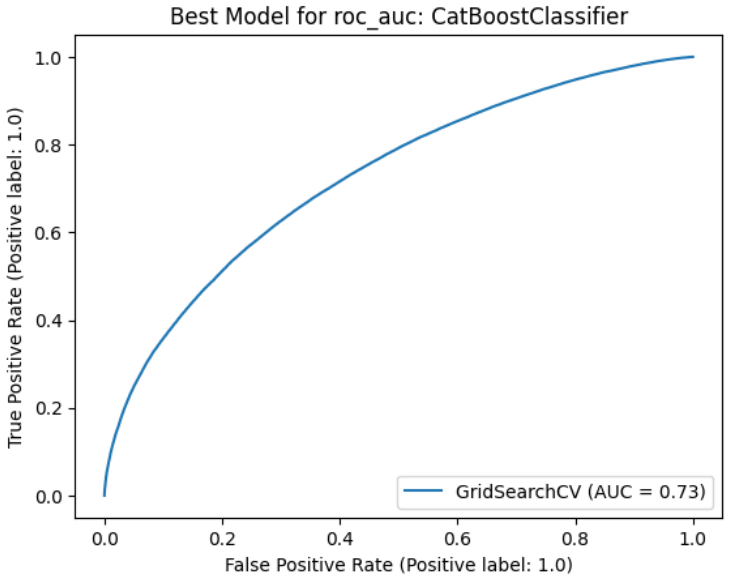
Best CatBoost params: `{depth: 6, iterations: 400, lr: 0.2}`.

EVALUATION

Final Model — Evaluation on the Test Set

The best model, **CatBoost**, is evaluated once on the isolated hold-out test set (~231K rows).





0.73
ROC-AUC

0.68
Accuracy

0.63
Recall

Takeaway: Performance is modest but clearly above random (AUC 0.50); the minority "no diabetes" class is the harder one to capture

CONCLUSIONS

Key Takeaways

Best Model

CatBoostClassifier achieved **ROC-AUC = 0.726** — modest but well above random (0.50), consistent with weak per-feature correlations found in EDA.

Boosting Wins

CatBoost, LightGBM, and XGBoost clustered tightly at **0.725–0.726**, clearly outperforming all other approaches.

No Class Weighting Needed

Top models used `auto_class_weights: None` — stratification alone handled the mild imbalance.

Scaling Matters

StandardScaler consistently outperformed RobustScaler and no-scaling, though differences were small for tree-based models.